RC Implementation Guide

v5.0.3 · Spektrum DX8 → AR6200 → Arduino Nano → Tesla rack · 17 May 2026

1. What v5 does

v5 keeps the v4.3.3 program (*tesla_control.py*) exactly as it was. It adds a sibling program *tesla_control_rc.py* and an Arduino Nano firmware (*arduino/tesla_rc_bridge/*) that lets you steer the Tesla rack from a Spektrum DX8 transmitter. Inside the laptop, the RC variant subclasses the v4.3.3 program and imports the existing CAN worker, all safety guards, and the session logger verbatim --the wire-level protocol is unchanged.

Steering math follows openpilot's *tools/joystick/joystickd.py* pattern: runtime min/max calibration of the stick endpoints, a 3% normalized deadband, and a cubic-blend expo curve ($0.4 \cdot x^3 + 0.6 \cdot x$). This is the same shape commaai ships in production for joystick control. Result: small stick movements give small wheel movements (for parking-lot corrections), large movements reach full lock-to-lock (±360°).

2. Hardware list

Item	Purpose	Source
Spektrum DX8 transmitter (any gen)	Operator radio	owned
Spektrum AR6200 receiver (SPMAR6200)	Receives DSMX, emits 6 PWM	owned
Arduino Nano (ATmega328P)	PWM → USB serial bridge	bench supply
3× servo lead wires + 2× jumpers	Rx → Nano	bench supply
Mini-USB cable	Nano → laptop	bench supply
SYS TEC USB-CANmodul1 (model 3204001)	Laptop → car CAN	unchanged from v4

3. Binding the AR6200 to the DX8

DX8 transmits DSMX with DSM2 fallback. AR6200 is a DSM2 aircraft receiver. They bind. Reference: *Spektrum AR6200 user guide*, PDF at spektrumrc.com/ProdInfo/Files/SPMAR6200.pdf.

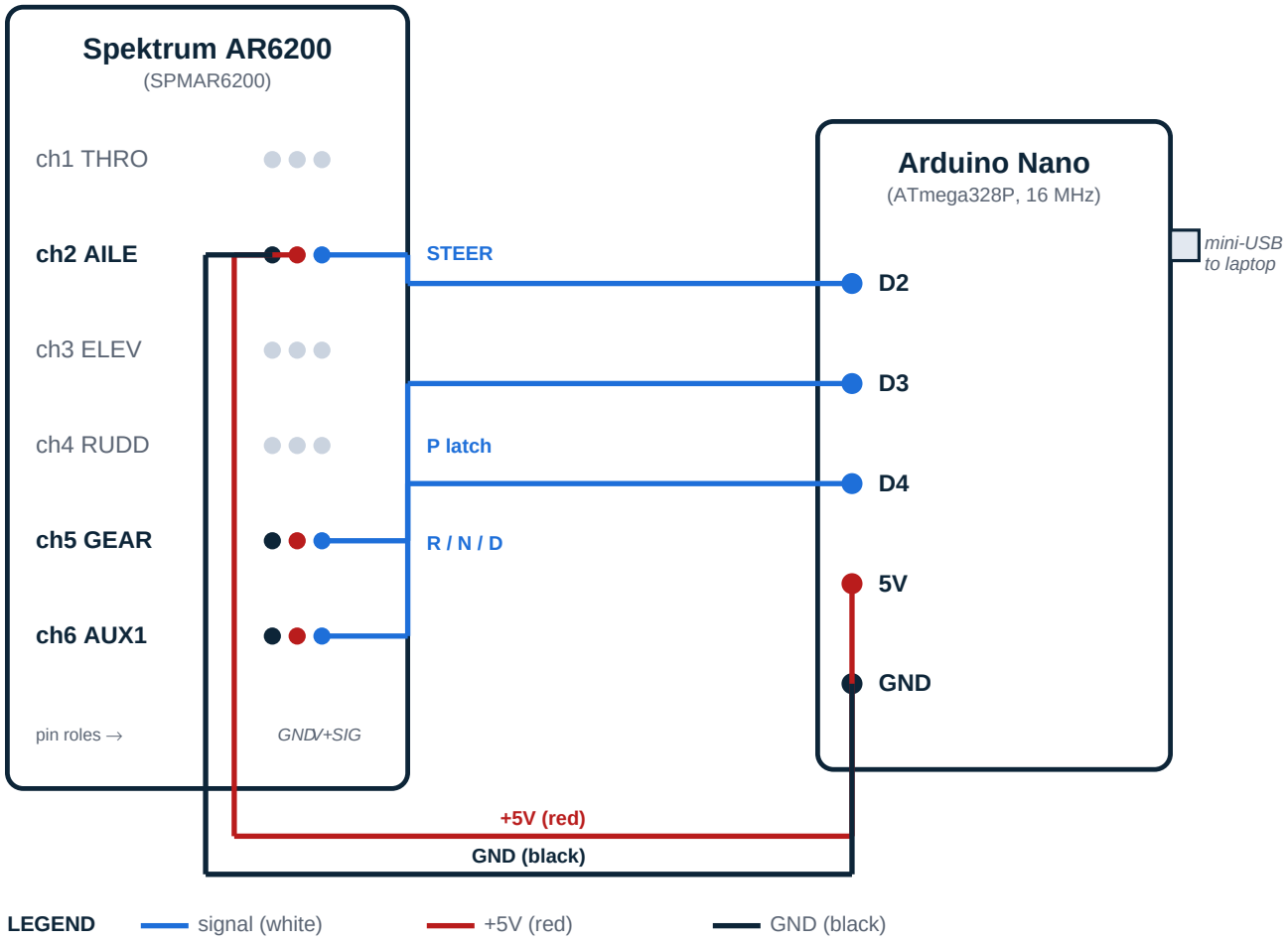
Step	Action
1	Insert the bind plug into the AR6200's BIND/DATA port.
2	Power the AR6200 (any servo port: GND + 5V is enough).
3	The receiver LED flashes rapidly while in bind mode.
4	On the DX8, hold the trainer / bind switch while powering on.

5	Within a few seconds the AR6200 LED goes solid -- bound.
6	Power-cycle the AR6200, remove the bind plug. Future power-ups auto-link.

If the LED keeps flashing past 30 s
The DX8 isn't in bind mode or the receiver isn't seeing it. Move closer, check antenna routing, retry. A re-bind never harms a receiver -- you can do it as many times as needed.

4. Wiring: AR6200 → Arduino Nano

Only three channels are wired into the Nano: **2 (AILE)** for steering, **5 (GEAR)** for the P latch, and **6 (AUX1)** for the R/N/D selector. The receiver is powered from the Nano's 5V regulated rail through channel 2's positive lead; channels 5 and 6 only need their signal wires.



5. Pin-by-pin table

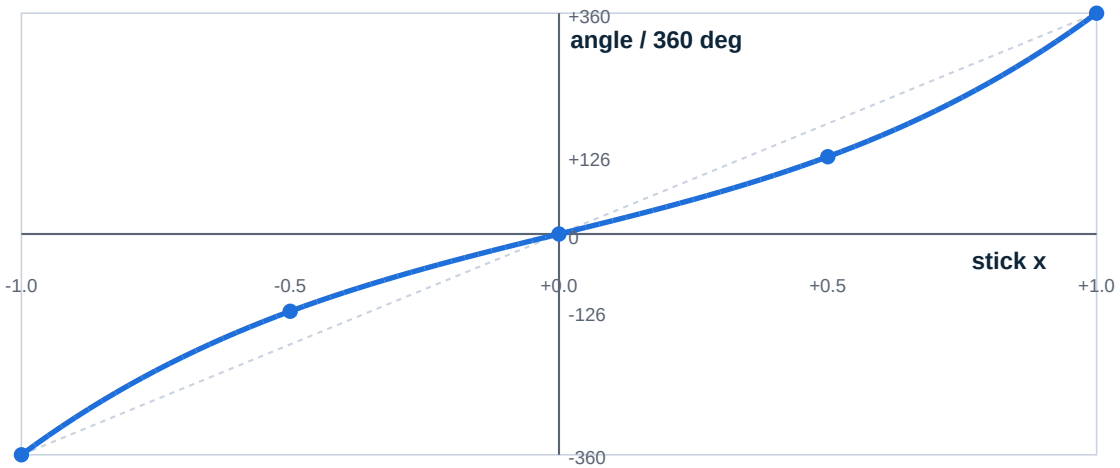
AR6200 channel & pin	Wire color (servo lead)	Goes to Nano pin	Carries
ch2 SIG (innermost)	white / orange	D2 (PCINT18)	Steering PWM
ch2 V+ (middle)	red	5V	Power to Rx
ch2 GND (outermost)	black / brown	GND	Common ground
ch5 SIG (innermost)	white / orange	D3 (PCINT19)	P-latch PWM
ch6 SIG (innermost)	white / orange	D4 (PCINT20)	R/N/D PWM

Channels 5 and 6 share power
Once channel 2 is wired up, the AR6200's internal rails energize every servo port. Channels 5 and 6 therefore only need their signal wires connected to the Nano. Tie a single GND between the receiver and the Nano, not three.

Do not double-power the AR6200
If the AR6200 is also connected to a separate battery (e.g. an ESC's BEC), DO NOT connect 5V from the Nano too. Tie grounds together, leave the V+ pin to the Nano disconnected. Otherwise you back-feed two regulators against each other.

6. Steering feel (expo curve)

The stick-to-angle mapping uses openpilot's exact cubic-blend expo from *tools/joystick/joystickd.py*. The shape is **linear-ish near center**, where you make small corrections, and **more aggressive near full deflection**, where you do parking maneuvers. Full stick travel reaches exactly ±HARD_ANGLE_LIMIT_DEG (±360° in v4.3.x).



openpilot expo: output = 0.4 x^3 + 0.6 x

Stick travel → wheel angle, full table

Stick position	Normalized x	Wheel angle
full left	-1.00	-360.0°
75% left	-0.75	-222.7°
50% left	-0.50	-126.0°
25% left	-0.25	-56.2°
10% left	-0.10	-21.7°
5% left	-0.05	-10.8°
centered	+0.00	+0.0°
5% right	+0.05	+10.8°
10% right	+0.10	+21.7°
25% right	+0.25	+56.2°
50% right	+0.50	+126.0°
75% right	+0.75	+222.7°
full right	+1.00	+360.0°

Why expo and not linear
Linear mapping wastes resolution at the center (small stick movements = noticeable wheel jumps) AND makes full lock feel twitchy. Expo gives you fine control where you spend 95% of your driving time, and reserves the last 25% of stick travel for the times you really need to crank the wheel.

7. PRND switch mapping (AUX1 + GEAR)

AUX1 (channel 6) is a 3-position switch on the DX8. The program reads the PWM width and selects D / N / R using these hysteresis bands. Switch DOWN selects drive, center is neutral, switch UP selects reverse.

PWM width (us)	AUX1 position	Gear
< 1250	switch DOWN	D (drive)
1250 to 1750	switch CENTER	N (neutral)
> 1750	switch UP	R (reverse)

A shift is dispatched only when the switch position **changes**: holding the switch in D does not continuously fire shift requests. The existing v4.3.3 non-blocking 200 Hz shift burst handles the SBW_RQ_SCCM transmission unchanged.

GEAR channel (channel 5) is a 2-position toggle configured as a momentary P button. The DX8 mapping is inverted from the AUX1 convention: **-100% travel (~1000 us) means PRESSED, anything at 0% or above (~1500 us+) means RELEASED**. Holding the button at -100% for 200 ms requests a shift to P. A 1-second cooldown prevents double-fire.

GEAR PWM (us)	DX8 reading	Meaning
<= 1250	-100% travel	P button PRESSED
1250 to 1500	deadband	transitioning
> 1500	0% or higher	P button RELEASED

8. Flashing the Arduino Nano

Install Arduino CLI from arduino.github.io/arduino-cli/installation, then:

```
cd arduino/tesla_rc_bridge
arduino-cli core install arduino:avr
arduino-cli compile --fqbn arduino:avr:nano:cpu=atmega328 .
arduino-cli upload --fqbn arduino:avr:nano:cpu=atmega328 -p .
```

Many Chinese-clone Nanos ship with the **old bootloader**. If upload fails with avrdude timeout, change the fqbn to *arduino:avr:nano:cpu=atmega328old*.

Finding the serial port

OS	Where to look
Windows	Device Manager → Ports (COM & LPT). 'USB Serial Device (COMx)' or 'CH340'.
macOS	ls /dev/cu.usbserial-* /dev/cu.usbmodem*
Linux	dmesg | tail -20 after plug-in. Usually /dev/ttyUSB0 or /dev/ttyACM0.

9. Running the program

Once the Nano is flashed and wired, the DX8 is bound, and the SYS TEC USB-CAN is plugged into both the laptop and the car's chassis CAN tap, run:

```
python tesla_control_rc.py --rc-port COM5
# macOS:
python tesla_control_rc.py --rc-port /dev/cu.usbserial-XXX
```

The GUI is identical to v4.3.3 plus a new **RC INPUT** strip near the top showing the live channel widths, the auto-calibrated stick range, current frame count, and the currently selected gear.

Calibration happens automatically. The first time you sweep the right stick fully left and right, the program records the actual endpoints and uses them from then on. Until that happens, the boot-default 1100..1900 us range is used (which works fine even uncalibrated -- calibration just tightens up the mapping for asymmetric trim).

Standard operating sequence

#	Action
1	Power DX8, power AR6200, plug Nano into laptop.
2	Plug SYS TEC into laptop USB and into the car's chassis CAN tap.
3	python tesla_control_rc.py --rc-port <PORT>
4	In the GUI, click CONNECT. Confirm the bus diagnostic panel populates.
5	Sweep the right stick fully left and right once to seed calibration.
6	Verify the RC INPUT panel shows STEER us tracking the stick.
7	Click ENGAGE. EAC transitions INHIBITED → AVAILABLE → ACTIVE.
8	Steer with the right stick. AUX1 switch picks D/N/R. GEAR toggle pulled to -100% = P.
9	ESC / Q / E-STOP button / close window = disengage.

Safety carries over from v4.3.3
Hard angle clamp at ±360°. Rate limit on the worker side at 150°/s. RX-timeout watchdog (500 ms with no 0x370 → E-STOP). EAC-bounce watchdog (>5 transitions/s → E-STOP). Real-motion auto-disengage when DI_vehicleSpeed > 1 mph. Park-to-engage gate. All of these still apply.

10. Signal-loss detection

Spektrum receivers **hold the last value** on transmitter power-off (SmartSafe). The AR6200 keeps emitting whatever PWM widths it last decoded, the Arduino keeps framing them, and the laptop keeps reading them. Without explicit detection, a dead transmitter looks identical to a stationary stick. v5 surfaces this in two ways:

Indicator	Trigger	What it means
NO SERIAL	No COBS frame for > 200 ms	Arduino died, USB stalled, or cable unplugged
TX LOST	Aileron PWM unchanged > 2 us for > 3 s	Spektrum holding last value; TX off / out of range
LIVE (green)	Frames arriving, stick changing	Normal operation

The SIGNAL pill in the RC INPUT panel shows the current state at all times. By itself the indicator does not interrupt operation -- you decide whether to E-STOP. For unattended or in-car testing, enable the **auto-disengage on signal loss** checkbox in the same panel. When checked, the program drops *ctrl.engaged* on the next UI tick if either SIGNAL condition fires. This is not an E-STOP -- you can re-engage once signal returns -- but it stops the rack from continuing to track a stale target.

Why not auto-disengage by default
On a bench setup with a flaky USB cable or a laptop that stutters its USB hub, the program would re-disengage every few seconds, which is more annoying than helpful. The checkbox lets you flip it on once the bench setup is known-good and you're moving to in-car testing where the consequence of a stuck stick matters more.

TX LOST is a heuristic, not proof
If you legitimately hold the stick perfectly still for 3 seconds, you'll see TX LOST. That's expected -- 3 seconds of perfect stillness is unusual on a real RC operator's stick. If you're driving slow and steady and trip the indicator, raise RC_FROZEN_STICK_TIMEOUT_S in the source.

11. Troubleshooting

Symptom	Likely cause	Fix
RC port FAILED to open	Wrong port / driver missing	Re-check Device Manager. CH340 driver if clone Nano.
STEER us stays at 0	White wire not on D2, or AR6200 not bound	Re-bind AR6200, re-check wiring against Section 4
STEER us frozen at 1500	TX off, receiver holds last	Power DX8 on. Spektrum holds last value on signal loss.
RND switch wobbles between R and N	Switch sits near a hysteresis edge	Increase EPA on AUX1 on the DX8 to push endpoints out
Shifts spam the log	PWM noise on AUX1	Increase hysteresis bands in tesla_control_rc.py
Stick travel doesn't reach ±360 deg	Calibration sweep not done	Push the stick fully left, then fully right, once
Wheel jitters at center	Dead band too tight for your stick	Increase RC_DEADBAND_NORM from 0.03 to 0.05
Wheel won't move past ~60 deg	Rack is in MIN_SPEED gating	Enable 30 MPH MODE in the GUI (jacked-up only)
SIGNAL pill shows TX LOST when stick is still	Hold stick perfectly still > 3 s	Normal heuristic; raise RC_FROZEN_STICK_TIMEOUT_S if too s
SIGNAL pill shows NO SERIAL intermittently	USB cable noise or hub power dip	Plug Nano directly into laptop USB, not through a hub

12. What v5 is NOT

Throttle, brake, and longitudinal control are NOT in v5. The pre-AP 2013 Model S has no CAN-commandable throttle and no iBooster -- adding those is the v6 scope (a pedal interceptor is required). The v5 RC bridge is steering-and-gear only.

v5 also does not replace the keyboard or slider modes. tesla_control.py v4.3.3 still runs exactly as before. v5 is an additional way to drive the same rack, not a replacement.

13. References

openpilot tools/joystick: github.com/commaai/openpilot/blob/master/tools/joystick/joystickd.py
Spektrum AR6200 manual: spektrumrc.com/ProdInfo/Files/SPMAR6200.pdf
Spektrum DX8 Gen 2 product page:
spektrumrc.com/product/dx8-8-channel-dsmx-transmitter-only-gen-2/SPMR8000.html
Consistent Overhead Byte Stuffing (Cheshire & Baker, 1999):
en.wikipedia.org/wiki/Consistent_Overhead_Byte_Stuffing
RCArduino multi-channel PWM read: rcarduino.blogspot.com/2012/04/how-to-read-multiple-rc-channels-draft.html
gregjhogan pre-AP EPAS patch: github.com/gregjhogan/tesla-pre-ap-epas-patch